

IC526 Advanced Compilers

Lec 10: LLVM IR I

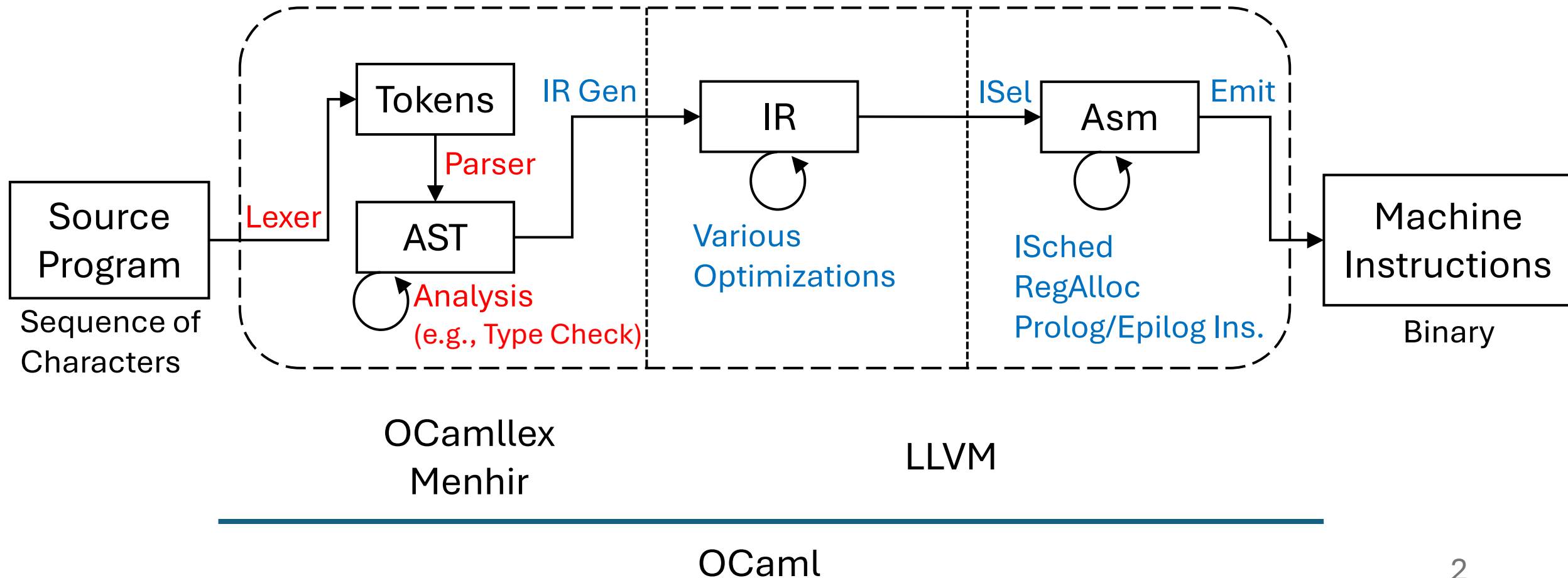
Yoonseung Kim

DGIST EECS

Fall 2026

Where We Are

- The final stage of the frontend: IR generation



Remaining Classes Before the Midterm

- LLVM IR I: Basics
 - Functions, Values, Types, Instructions, and Memory
- LLVM IR II: Advanced
 - Control flow, arrays, references, and noalias
- IR Generation
 - Using the LLVM API through its OCaml bindings
 - Translating the typed AST into LLVM IR

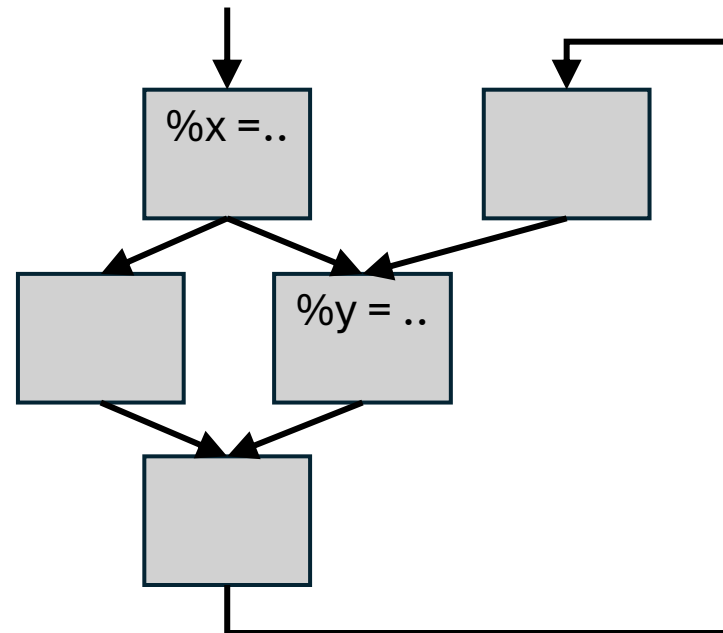
Introduction to LLVM & LLVM IR

LLVM and LLVM IR

- LLVM provides reusable libraries and tools for building compilers.
- LLVM IR is a low-level intermediate language used by LLVM.
- With LLVM, we can:
 - Construct and inspect LLVM IR.
 - Use existing analysis and optimization passes, or implement our own.
 - Generate machine code for different targets.

Key Features of LLVM IR

- **Typed values:** each value has a type.
- **Basic blocks:** control flow is expressed using blocks.
- **SSA values:** each virtual register has a single definition.
- **Explicit memory access:** instructions such as load and store read and write memory.



SSA = Static single assignment

Analysis and Transformation Passes

- **Analysis passes** compute information about a program.

Examples:

- Which functions call each other? (call-graph analysis)
- Can two pointers refer to the same memory location? (alias analysis)

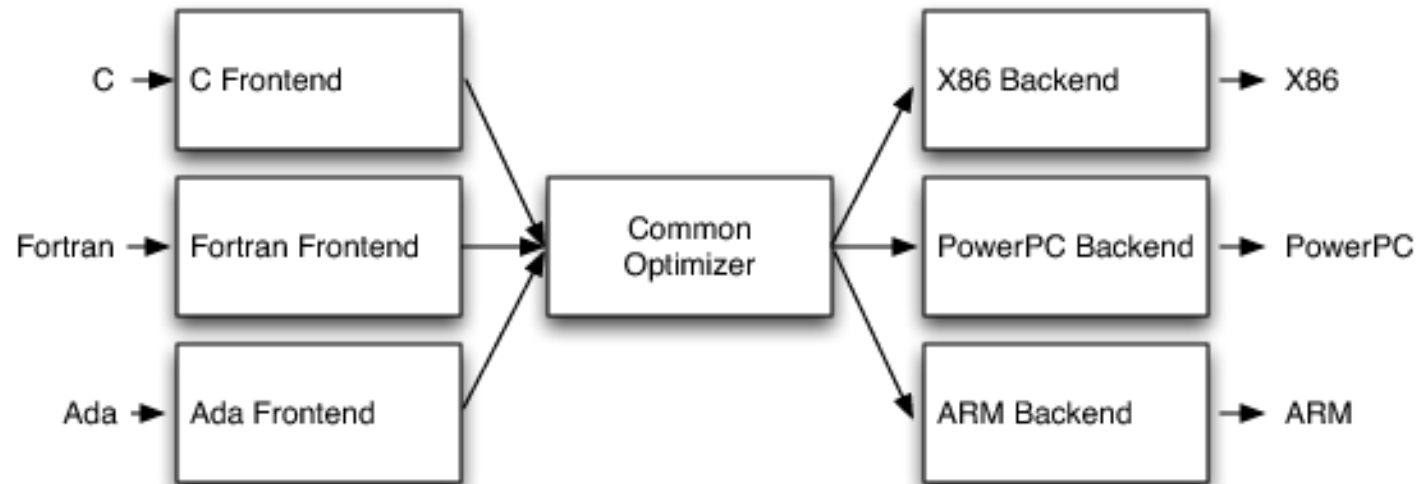
- **Transformation passes** modify the program.

Examples:

- Optimization: remove redundant or dead operations.
- Instrumentation: insert code to record execution counts or paths.

Targets and Code Generation

- LLVM supports multiple target architectures.
 - A **shared middle-end** optimizes LLVM IR for different targets.
 - **Each target backend** generates machine code for its target.
- Target information can also guide middle-end optimizations.
 - E.g., choosing profitable vector widths.



Working with LLVM IR

- LLVM IR has three representations:
 - **Textual IR (.ll)**: a human-readable representation.
 - **Bitcode (.bc)**: a binary encoding of LLVM IR.
 - **In-memory IR**: C++ objects constructed and manipulated through LLVM APIs.
- In this course, we will:
 - Read and understand textual LLVM IR.
 - Construct and transform IR using the OCaml bindings.

A First Look at LLVM IR

- RustY

```
fn add_one(x: i32) -> i32 {  
    return x + 1;  
}
```

- LLVM IR

```
define i32 @add_one(i32 %x) {  
entry:  
    %result = add i32 %x, 1  
    ret i32 %result  
}
```

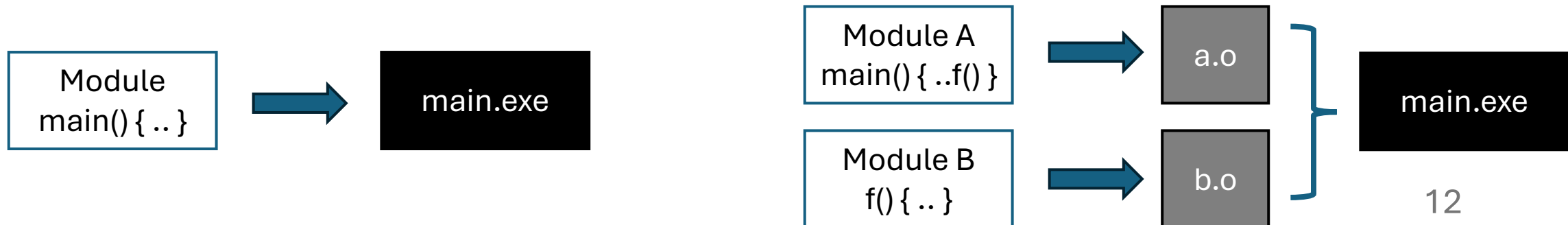
- entry: a basic block
- %x and %result: virtual registers

	Programming languages	Machine code	LLVM IR
Value storage	Variables	Physical registers & memory	Virtual registers & memory
Control flow	Structured (If, While, ..)	Jumps	Basic blocks

LLVM Modules

A Module

- A module is a unit of translation.
 - A .ll / .bc file is parsed to a single module.
 - In our project, a RustY source file forms a single module.
- A module contains global definitions and declarations.
 - Functions and global variables
 - A module may also contain target information and metadata.
- A program is composed of (one or more) modules.
 - A complete program has one definition of main



Global Definitions and Declarations

- **Function definitions** provide function bodies.
- **Global variable definitions** provide storage and initial values.
 - **global:** the contents can change.
 - **constant:** the contents cannot change.
- **Declarations** introduce functions or global variables defined elsewhere.

Module: An Example of .ll code

```
; Global declarations
declare i32 @external(i32)
@external_var = external global i32
@external_const = external constant i32

; Global definitions
; @gvar and @gconst denote addresses of global memory objects
@gvar = global i32 0
@gconst = constant i32 100

define i32 @sum_globals() {
entry:
    %c = load i32, ptr @gvar ; read the stored value into %c
    %l = load i32, ptr @gconst
    %sum = add i32 %c, %l ; add the loaded values
    ret i32 %sum
}
```

Names in LLVM IR

@inc	function name
@message	global variable name
%x	parameter or instruction result
%entry	basic block operand

A block definition writes its label as "entry:".

Types and Operations

Types Used by RustY

RustY	LLVM IR
i32	i32
usize	i64
bool	i1
unit return	void
[i32; 4]	[4 x i32]
&i32 / &mut i32	ptr

usize uses 64 bits for our target.

LLVM also supports many types outside this course.

Integer Types

An integer type iN describes an N -bit value.

$i1$ 1 bit

$i32$ 32 bits

$i64$ 64 bits

The instruction chooses signed or unsigned behavior.

$11111111_2 = -1$ (signed) $= 255$ (unsigned)

Instruction Syntax

```
%sum = add i32 %x, 3
```

result	opcode	operand type	operands
%sum	add	i32	%x, 3

Both operands have type i32. The result also has type i32.

```
%ok = icmp slt i32 %x, 3
```

icmp compares i32 operands and produces an i1 result.

Integer Arithmetic

`%a = add i32 %x, %y`

`%b = sub i32 %x, %y`

`%c = mul i32 %x, %y`

The operand types must agree.

`; %x = 11111010 (-6 or 250)`

`%s = add i8 %x, 2 ; -4 (11111100)`

`%u = add i8 %x, 2 ; 252 (11111100)`

For i64 values, use the same opcodes with i64.

`%d = add i64 %n, 1`

Signed and Unsigned Operations

Source type	Division	Remainder
i32	sdiv	srem
usize	udiv	urem

```
; %x = 11111010 (-6 or 250)
%s = sdiv i8 %x, 2    ; -3  (11111101)
%u = udiv i8 %x, 2    ; 125 (01111101)
```

The source type determines which opcode we select.

Type Annotations Guide IR Generation

```
fn divide(x: i32, y: i32) -> i32 {  
    return x / y;  
}
```

```
%r = sdiv i32 %x, %y
```

```
fn divide_u(x: usize, y: usize) -> usize {  
    return x / y;  
}
```

```
%r = udiv i64 %x, %y
```

Integer Comparisons

Meaning	i32	usize
<code>equal(==)</code>	<code>eq</code>	<code>eq</code>
<code>not equal(!=)</code>	<code>ne</code>	<code>ne</code>
<code>less than(<)</code>	<code>slt</code>	<code>ult</code>
<code>less or equal(<=)</code>	<code>sle</code>	<code>ule</code>

```
%a = icmp slt i32 %x, %y
```

```
%b = icmp ult i64 %n, %limit
```

Each comparison produces i1.

For greater comparisons, use `sgt/ugt` or `sge/uge`.

Bitwise Operations

`%both` = `and i1 %a, %b`

`%either` = `or i1 %a, %b`

`%neg` = `xor i1 %a, true`

For `i1`, `and` / `or` implement conjunction / disjunction.
`xor` with `true` implements negation.

Our RustY `&&` and `||` evaluate both operands.

Exercise: Selecting Instructions

Assume `x, y: i32` and `n, limit: usize`.

1. `x + y`
2. `n / limit`
3. `x < y`
4. `n < limit`

For each expression, choose an opcode, an operand type, and a result type.

Functions

Function Types

```
define i64 @f(i32 %x, i64 %n) {  
  entry:  
    ret i64 %n  
}
```

$f: (i32, i64) \rightarrow i64$

The signature determines parameter and return types.

A declaration gives a signature without a body:

```
declare i64 @f(i32, i64)
```

Function Calls

```
declare i64 @f(i32, i64)

define i64 @caller() {
entry:
    %r = call i64 @f(i32 3, i64 4)
    %s = add i64 %r, 1
    ret i64 %s
}
```

A call specifies the return type and each argument type.

The call result is an SSA value (=virtual register) that later instructions can use.

Functions with No Return Value

```
declare void @notify(i32)

define void @run() {
entry:
    call void @notify(i32 3)
    ret void
}
```

A void call has no SSA result to name.

ret void ends a function that returns no value.

RustY uses this form for a unit-returning function.

Memory Operations

Values and Addresses

```
let mut x: i32 = 10;
```

The variable has a storage location and a stored value.

```
%x.addr = alloca i32  
store i32 10, ptr %x.addr
```

```
...
```

```
%v = load i32, ptr %x.addr
```

%x.addr is a ptr value. %v is an i32 value.

alloca

```
%x.addr = alloca i32
```

```
%a.addr = alloca [4 x i32]
```

alloca reserves a memory slot for the requested type.

Its result is a pointer to that slot.

The memory slot lasts until this function invocation returns.

alloca does not initialize the stored value. (the “undef” value)

load and store

```
store i32 10, ptr %x.addr  
%a = load i32, ptr %x.addr  
store i32 20, ptr %x.addr  
%b = load i32, ptr %x.addr
```

store changes memory and has no result value.

load reads memory and produces a new SSA value.

Here, %a is 10 and %b is 20.

Source Mutability

```
let x: i32 = 10;  
let mut y: i32 = 20;
```

The typechecker decides which assignments are legal.

Our IR generator gives both variables memory slots.

An immutable source binding can still have an alloca

An Assignment in LLVM IR

```
x = x + 1;
```

```
%old = load i32, ptr %x.addr  
%new = add i32 %old, 1  
store i32 %new, ptr %x.addr
```

Read the old value, compute the result, then write it.

The address stays the same while its contents change.

SSA and Mutable Memory

Static single assignment (SSA):
each named result has one defining instruction.

```
%a = load i32, ptr %x.addr  
store i32 20, ptr %x.addr  
%b = load i32, ptr %x.addr
```

%a and %b are distinct values. Neither is reassigned.

Memory can change even though SSA values do not.

Allocation and Initialization

Our IR generator places local allocas in the entry block.

The initializer runs where the declaration executes.

```
while condition {  
    let x = f();  
    ...  
}
```

The slot for x is allocated once per function call.

The call to f and its store run on each loop iteration.

Exercise: Translating a Function

```
fn step(x: i32) -> i32 {  
    let mut y = x + 1;  
    y = y * 2;  
    return y;  
}
```

Use a slot for y. You may use parameter %x directly.

Write a definition with one entry block.

Choose a fresh name for each intermediate result.

One Translation

```
fn step(x: i32) -> i32 {  
    let mut y = x + 1;  
    y = y * 2;  
    return y;  
}
```

```
define i32 @step(i32 %x) {  
entry:  
    %y.addr = alloca i32  
    %init = add i32 %x, 1  
    store i32 %init, ptr %y.addr  
    %old = load i32, ptr %y.addr  
    %new = mul i32 %old, 2  
    store i32 %new, ptr %y.addr  
    %result = load i32, ptr %y.addr  
    ret i32 %result  
}
```

Summary

LLVM instructions operate on typed values.

The source types determine widths and signed operations.

SSA names identify values. load and store access memory.

Functions specify parameter types and a return type.

Next: branches, loops, arrays, and reference parameters.